Device Groups

Tags	master_logs.system_runs, process runtimes, master_dev.device_groups_rules, Groups, dump_dgr_tables.sh, event suppression, refresh_rule_selector_types, does not have a populated rule query, device_group_analyzer.py
Team Owner	Proc and Roll
Status	ACTIVE
Last Modified	Dec 22, 2022

- Introduction
  - Terminology
  - Basic Architecture
- Diagnosis and Mitigation
  - How long are dynamic device updater runtimes
  - set TTL to 60
  - Dumping Rules for Later Review from a Customer System
  - Measuring rule id run times
  - Using seed devices and DCM or DCM+R relationships to define rules
  - Slow Device Group Editor and AWS: a possible reason in versions before 10.2.4
  - Slow Rule Evaluations: clean up unused extended attributes
  - finding all of the dids in a given dynamic device group from the cache
- Theory
  - Strategy
  - Tutorial Exercise
  - Tables
    - master_dev.device_groups
    - master_dev.device_groups_to_devices
    - master_dev.device_groups_to_groups
    - master_dev.device_groups_rules
    - master_dev.device_groups_rules_selectors

        - master_custom.ent_device_custom_extended_string
        - master_custom.ent_device_custom_extended_int
        - master_dev.device_groups_rules_depth_expansion
        - dynamic_rules.rule_dids_X
        - master.definitions_rule_selector_types
        - master.V_rule_selector_types.sql
    - stored procedures
        - master_dev._get_rule_dids_execute_cached()
    - dynamic device updater process
- Post Mortem Articles
    - 2024-08-22 EM-66517: MF rows behind fluctuating between 500k and 600k, found 4 DROP TEMPORARY TABLE IF EXISTS queries stuck for a long time that could be the cause of rows behind
    - 2024-08-22 EM-66519: Parent device groups made up of dynamic child groups only show the first page of group members
    - 2024-08-15 EM-66436: Device Group ID xxx does not have a populated rule query (rule ID xxx)
    - 2024-07-24 EM-66104: "Dev Count" column on Device Groups page is not accurate
    - 2024-04-23 EM-64567: Incorrect devices listed for Group-UI view and API Get Output
    - 2024-03-27 EM-64038: Dynamic Device group contains incorrect devices
    - 2024-03-27 EM-64031: Substantial amount of long running SQL Queries in SLEEP and Locked Tables
    - 2024-03-15 EM-63830 Query 9972328706 has been active for 290292 seconds (Threshold 900)
    - 2024-02-22 EM-63460: Device Grouping and Event Suppression
    - 2024-02-21 EM-63455: Event suppressions occactionally not working correctly
    - 2024-02-21 EM-63432: events generating for devices in suppression (Dynamic device) group
    - 2024-01-25 EM-63055: Users losing access to device groups unexpectedly.
    - 2022-04-04 PQR BV Device Group Properties Page Timeout
    - 2021-09-12 Chubb and Son: master_dev.get_assoc_group_dids_literal() called by UI with 100s of DGIDs to evaluate
- Known Issues
    - deadlocks with multi-row legend device updaters
- Silo Log Messages
        - master_dev.device_groups_to_devices
    - API doc

Introduction

Dynamic device groups are a particularly challenging feature to troubleshoot, due both to their innate complexity as a concept, and due to most of the membership of a group being determined in MySQL stored procedures. A user's own device group composition, load on the database appliance, and platform (mariadb, Aurora) serving the SL1 database all have different impacts on evaluating and presenting device groups. A large amount of scale testing and resulting changes to the SQL code used for device groups was made in SL1's v11.1.1.2 hotfix and resulting releases/MRs, as well as a design change around how custom attributes were being used, in the v12.1.0 release.

Terminology

A device group is a collection of devices within the SL1 database that a user typically wants to aggregate activity of and view data across. A device group can contain static or dynamic members. A static member is one that was specifically added to the group without any dynamic

rules used to include it. A dynamic member is a device included because it matches one or more **dynamic rules**. These dynamic rules themselves consist of one or more **selectors**, of which there are two primary categories: direct, and topology. Often times, device groups end up being referred to as **dynamic device groups**, as membership is most often included via dynamic rules. Because device group membership is requested quite often by a lot of different SL1 processes running at the same time, there is a caching design around such membership evaluation. By default, every 5 minutes, each dynamic rule defined is reevaluated, and device IDs which pass the rule are put into its own isolated table as a list of matches. Requests for group membership, then, involves returning the union of all rules defined in the group, to the requestor. The 5-minute default time is referred to as the Time-To-Live (TTL), which will be covered further below.

A **direct selector** is something that selects directly against a property of the device, such as name, IP, device state, etc. A single rule may have one or more direct selector, and we evaluate the **conjunction** (x AND y AND z) of all direct selectors to include matches that pass all selectors in the rule.

A **topology selector** can be assigned to a rule which already has at least one direct selector. A topology selector allows us to modify the direct selectors to instead select related devices based on DCM+R or network topology relationships. These include parents, children, all ancestors, all descendants, seed devices (ie, the devices which were selected by the direct selectors), and relationships from specific apps. We evaluate the **disjunction** (i OR j OR k) of all topology selectors - if, for example, you have one for children, and one for parents, we're selecting all parents and all children - not devices which are both parent and child to the seed device.

Just as a dynamic rule is built from the conjunction of one or more direct selectors (and the disjunction of associated topology selectors), a dynamic group is built from the disjunction of one or more dynamic rules. So if you include 2 or more rules in a device group, the union of device matches across each of the rules will be included as members of the group.

Basic Architecture

The large majority of device group evaluation is done via MySQL stored procedures, defined in device_group_procedures.sql (or procedures.sql prior to the 11.3.0 release). These procedures are called either from the Dynamic Device Groups Updater SL1 process (of which a new instance runs every minute) or from the Device Group Editor.

In the database, you'll want to pay attention to the following tables:

master_dev.device_groups: This stores the basic device group definitions

master_dev.device_groups_rules: This maps rules to groups, and it is where you define the TTL for a rule - the length of time, in minutes, before it's automatically recalculated and cached. The last_calculated field is also of interest, since that's the timestamp against which we're evaluating the TTL.

master_dev.device_groups_rules_selectors: This contains the definitions for direct selectors. Do not expect to be able to read this, but you do need to know of its existence.

master_dev.device_groups_rules_depth_expansion: This contains definitions for topology selectors. Don't expect to be able to read this, either, but you do need to know about it.

dynamic_rules.rule_dids_<rule_id>: This is the cache table for a given rule ID. It's repopulated via dynamic_device_groups.py, which invokes master_dev._get_rule_dids_execute_cached, which in turn evaluates the provided rule ID and stores the results in the cache.

Besides these tables, you also need to be aware of the stored procedures which we use. The most important stored procedure you'll be working with is **master_dev._get_rule_dids_execute()**, and its sibling, **_get_rule_dids_execute_cached()**. These are where the large majority of calculations are kicked off (although quite a bit is done in additional procedures/functions which are called from here). There are many additional functions like **get_rule_dids**, **get_rule_count**, and **get_multi_rule_dids**, but all of these ultimately make use of get_rule_dids_execute. The only difference between _execute() and _execute_cached() is that the latter stores results in dynamic_rules.rule_dids_<rule_id>, while the former stores in master_dev._get_rule_dids_result.

master_dev.get_rule_dids_execute() takes 4 arguments: rule, restrict_to_subset, restrict_to_user, and clear_first. Generally speaking, you can leave the values of the last three as 0, 0, and True when attempting to manually invoke this function. The second and third arguments are for multitenancy, which doesn't give us any trouble here, and clear_first is to make sure that the output table is empty before we start inserting to it.

These pages have a lot of detail on how dynamic device groups work

- Dynamic Device Group Troubleshooting. [ToDo: Delete this link, the page it points to, and replace its location with a link to this file here]
- Dynamic Device Group Architecture (as of 10.1.0)
- Dynamic Device Group Interaction Guide (as of 10.1.0)

Diagnosis and Mitigation

How long are dynamic device updater runtimes

By running this query

```
silo_mysql -e "select * from master_logs.system_runs where sp_id in (162) and start_time>=now() - interval 1 day order by start_time" > system_runs.txt
```

you can check the runtimes of the dynamic device updater. When dynamic device group rules are badly optimized, it will show up in long runtimes for the updater.

set TTL to 60

By running this query on the CDB:

```
UPDATE master_dev.device_groups_rules SET TTL=60;

```

You can mitigate the effect of expensive/slow dynamic device group rules temporarily by doing this. The TTL, or time to live, for a query, is the number of minutes between executions of the rule to assess dynamic group membership. This query sets the TTL for all rules to 60 minutes. The updater runs every minute so if, for example, a badly optimized rule were causing a single run of the updater to take 20 minutes to run, you would expect to have around 20 instances of the updater running at the same time. As some of these rules can imply queries that hold locks, this can cause a lot of contention on the DB.

Once you have set the TTL to 60, the next step would be to examine the individual rules and see which ones are taking a long time to run.

Dumping Rules for Later Review from a Customer System

Running this script on the CDB:

dump_dgr_tables.sh
16 Jul 2024, 02:48 PM

will

- gather device group tables and
- system runs data for the specified time period
- by default this is '1 day'

- you can specify other time periods like this (you have to quote the time expression):

```
./dump_dgr_tables.sh '2 day'
./dump_dgr_tables.sh '4 hour'

```

Its also a good idea to run dump_cdr_tables.sh by following the instructions here: Rows-Behind Troubleshooting Guide | Gathering Table Dumps: dump_cdb_tables.sh

as the tables dumped by that script are often useful in analyzing problems with device groups.

By going through the Tutorial Exercise section below, you can see how a rule is encoded into those tables.

Measuring rule id run times

if you turn on debug logging for the DDGU process, and capture a few hours of /data/silo.log on the CDB and then look at these debug messages:

```
    def executeStoredProcedure(self, which, rule):
        retVal = None
        if which in self.storedProcedures:
            retVal = True
            try:
                self.dbc.execute_d("UPDATE master_dev.device_groups_rules SET last_calculated=\
                                   NOW() where rule_id=%s", rule)
                sql = "CALL %s(%s)" % (which, rule)
                self.logger.debug("Calling Stored Procedure: %s" % (sql))
                self.dbc.execute_d(sql)
            except Exception as e:
                self.logger.debug("Failed to execute rules procedure %s for rule ID %s"
                               % (self.storedProcedures[which], rule))
                errorCode = None
                errorMessage = None
                try:
                    errorCode = e[0]
                    errorMessage = e[1]
                except:
                    errorCode = None
                    errorMessage = e[0]
                self.logger.debug("Error Code: %s, %s" % (errorCode, errorMessage))
                retVal = False

        return retVal

```

you could subtract timestamps for this message

```
                self.logger.debug("Calling Stored Procedure: %s" % (sql))

```

to determine how long each instance of each rule id took to run.

I have written this python program to do this analysis: https://code.eng.sciencelogic.com/projects/TOOL/repos/em7_diag/browse/analysis_examples/dynamic_device_groups/analyze_ddgu_timing.py

As the docstring at the top of the program explains:

```
"""
This program finds lines like these from a silo.log:
    "2022-04-07 12:00:33,878","DEBUG","64409","162","dynamic_device_groups","83","Calling Stored Procedure: CALL master_dev._get_rule_dids_execute_cached(175)"

and subtracts adjacent rule executions to produce a CSV report of : rule_id, time executes (seconds since epoch), runtime.  For example:

175,1649332833.878,7.065999984741211
177,1649332840.944,6.523999929428101
180,1649332847.468,6.493000030517578
183,1649332853.961,7.26200008392334
185,1649332861.223,8.407999992370605
195,1649332869.631,7.174999952316284

if you ran this program like this:
./analyze_ddgu_timing.py < silo.log > run_times.txt

You could plot the Cumultive Distribution Function in an R session with:
    df <- read.csv("rule_times.txt", header=FALSE, col.names= c("rule_id","t","dt"))
    plot(ecdf(df$dt), main=paste("CDFs of rule executions (mean(dt)=",mean(df$dt),")"), xlim=c(0,max(df$dt)))
"""

```

This case is an example where this analysis was done: PQR BV Device Group Properties Page Timeout | 2022 04 07 Analysis: most rules take around 7 seconds to execute

Using seed devices and DCM or DCM+R relationships to define rules

Following the click path registry → devices → device groups → create takes you to an editor panel with three vertically stacked frames

- top frame: name and other attributes
- middle frame: static devices and groups
- bottom frame: dynamic rules SLUI-7126: GQL/AP2 - Add pagination to cleared events CLOSED

From the bottom frame, if you click add, you enter a dialog with three frames

- a list of active selectors and checkboxes
- selector definitions: an area where additional controls appear as you enable selectors
- matched devices: this area holds a list of devices, initially empty, that fills with devices as you enable selectors and configure them to match existing devices

If you wanted to add a particular component device, and all of its descendants, into the dynamic device group rule you could

- enable the device name selector in frame 1 and specify a pattern that matced one you were interested in
- once that device appeared in frame 3, you could click on "select related devices" in the lower right corner of frame 3 and then
- select these three checkboxes now available in frame 1: children of, descendants of, and include seed devices.

At this point, your original component and its descendants would all be visible in the list of included devices.

Slow Device Group Editor and AWS: a possible reason in versions before 10.2.4

As Nicholas Rios (The Forgotten Coders) pointed out in EM-50089:

```
The device group editor gives us problems in AWS because it uses a lot of temp tables, in 10.2.4 we reduce the
creation of that temporal tables so should greatly improve the times
```

Slow Rule Evaluations: clean up unused extended attributes

In old versions of EM7, the device deleter did not delete the custom attributed associated with the device being deleted. The device deleter was fix to handle this in EM-50073.

This may solve the problem all by itself.

Here is a case where it did: PQR BV Device Group Properties Page Timeout

As Nicholas Rios (The Forgotten Coders) pointed out in EM-50089:

```
There are other issues we discover that affect the times for calculating rules related to the use of a table view (master.V_rule_selector_types)
getting the results of this can take a good amount of time depending on the size of the tables like master_custom.ent_device_custom_extended_string
even if the rule doesn't use custom attributes

for this, some changes were made in 11.1.1.2, I am not sure of the upgrade plan this customer has so don’t know when can reach this version

there is other work that is being done for 11.2.0, 11.1.2 so for the moment I don’t think you are going to have access to it.

and for that, I will recommend also running the script to remove the rows of ent_device_custom_extended_string:

CREATE TEMPORARY TABLE IF NOT EXISTS master_dev.tmp_custom_attr_dids (
  did INT UNSIGNED NOT NULL,
  PRIMARY KEY (did)
) ENGINE=MEMORY;
```

```
INSERT IGNORE INTO master_dev.tmp_custom_attr_dids SELECT id FROM master_dev.legend_device;
INSERT IGNORE INTO master_dev.tmp_custom_attr_dids SELECT id FROM master_dev.legend_device_vanished;
INSERT IGNORE INTO master_dev.tmp_custom_attr_dids SELECT id FROM master_dev.legend_device_del;
DELETE FROM master_custom.ent_device_custom_extended_string WHERE ent_id NOT IN (SELECT did FROM master_dev.tmp_custom_attr_dids) LIMIT 500000;
DROP TEMPORARY TABLE IF EXISTS master_dev.tmp_custom_attr_dids;
```

So, even if you have a simple rule that is only checking for the devices aligned with a given dynamic app, the fact that the stored procedures use the view master.V_rule_selector_types could slow that rule's execution down if the table master_custom.ent_device_custom_extended_integer is large.

I believe this multi delete query would also do the job and might be faster:

```
use master_custom;
delete mcedcs from master_custom.ent_device_custom_extended_integer as mcedcs
left join master_dev.legend_device as ld on (ld.id=mcedc.ent_id)
left join master_dev.legend_device_vanished as ldv on (ld.id=mcedc.ent_id)
left join master_dev.legend_device_del as ldd on (ld.id=mcedc.ent_id)
where ld.id is NULL and ldv.id is NULL and ldd.id is NULL
limit 500000;
```

and you could use a variation on this to first see how bad the problem is (how many rows would be deleted):

```
select count(*)
from master_custom.ent_device_custom_extended_string as mcedcs
left join master_dev.legend_device as ld on (ld.id=mcedcs.ent_id)
left join master_dev.legend_device_vanished as ldv on (ld.id=mcedcs.ent_id)
left join master_dev.legend_device_del as ldd on (ld.id=mcedcs.ent_id)
where ld.id is NULL and ldv.id is NULL and ldd.id is NULL;
```

finding all of the dids in a given dynamic device group from the cache

There is no single query that will do this because, however, here is how you would do it conceptually. If you want to find, from the dynamic device group cache, all of the dids belonging to group X, you would

```
select rule_id as Y from master_dev.device_group_rules where owner_type=1 and owner=X;
for each rule_id Y:
   select did from dynamic_rules_<Y>;
```

Here is how this bash script:
https://code.eng.sciencelogic.com/projects/TOOL/repos/em7_diag/browse/dump_dgr_tables.sh

arranges to dump that information in text form:

```
silo_mysql -e "select table_name from information_schema.tables where table_schema='dynamic_rules'" | grep -v table_name | while read table; do
   echo "dumping $table"
   silo_mysql -e "select * from dynamic_rules.${table}" > dr_${table}.txt
```

```
done
```

Theory

Strategy

This document describes high level strategy for tuning device group rules: [Dynamic Device Group Troubleshooting]

Tutorial Exercise

Follow these instructions on a test stack, and you will have a better idea of how the tables around dynamic device groups fit together.

```
# FIXME_INFO_ISSUE_3_1: what would an example of the use of dynamic device groups look like?
# FIXME_INFO_ISSUE_3_2: we create three virtual devices with these names: vdev_type_a_type_b, vdev_type_b_type_c, vdev_type_c_type_a.  Our strategy to demonstrate dynamic rules will be to write a named based rule with a mysql search pattern of 'vdev_%type_a%' and another with a pattern of 'vdev_%type_c%'.
# FIXME_INFO_ISSUE_3_3: after creating the virtual devices, our test stack has this device table:
select id, device from master_dev.legend_device;
+----+--------------------+
| id | device             |
+----+--------------------+
|  1 | cdb                |
|  2 | vdev_type_a_type_b |
|  3 | vdev_type_b_type_c |
|  4 | vdev_type_c_type_a |
+----+--------------------+
# FIXME_INFO_ISSUE_3_4: using registry -> devices -> device groups -> create, we create
# FIXME_INFO_ISSUE_3_5: a group called group_vdev with static members dids 2, 3, 4.
# FIXME_INFO_ISSUE_3_6: a dynamic group called group_vdev_type_a, with a name based selector of: 'vdev*type_a*'
# FIXME_INFO_ISSUE_3_7: a dynmaic group called group_vdev_type_c, with a name based selector of: 'vdev*type_c*'

# FIXME_INFO_ISSUE_3_8: so we see groups 12, 13, and 14
MariaDB [(none)]> select name, dgid from master_dev.device_groups where dgid>11;
+-------------------+------+
| name              | dgid |
+-------------------+------+
| group_vdev        |   12 |
| group_vdev_type_a |   13 |
| group_vdev_type_c |   14 |
+-------------------+------+

# FIXME_INFO_ISSUE_3_9: we can see the static membership of group_vdev like this:
select * from master_dev.device_groups_to_devices where dgid=12;
+------+-----+
| dgid | did |
+------+-----+
|   12 |   2 |
|   12 |   3 |
|   12 |   4 |
```

```
+------+-----+

# FIXME_INFO_ISSUE_3_10: In defining group_vdev_type_a and group_vdev_type_c we created two rules
MariaDB [(none)]> select * from master_dev.device_groups_rules where rule_id>37;
+---------+------------+----------+-----+---------------------+
| rule_id | owner_type | owner_id | TTL | last_calculated     |
+---------+------------+----------+-----+---------------------+
|      38 |          1 |       13 |   5 | 2020-07-14 22:00:10 |
|      39 |          1 |       14 |   5 | 2020-07-14 22:00:07 |
+---------+------------+----------+-----+---------------------+

# FIXME_INFO_ISSUE_3_11: with two selectors
MariaDB [(none)]> select * from master_dev.device_groups_rules_selectors where rule_id>37;
+---------+-----------------+---------------+-------------------+-----------------+-------------------+----------+
| rule_id | selector_domain | selector_type | selector_instance | selector_target | selector_pattern | inverted |
+---------+-----------------+---------------+-------------------+-----------------+-------------------+----------+
|      38 |               0 |             3 |                 0 |               0 | vdev%type\_a%    |        0 |
|      39 |               0 |             3 |                 0 |               0 | vdev%type\_c%    |        0 |
+---------+-----------------+---------------+-------------------+-----------------+-------------------+----------+

# FIXME_INFO_ISSUE_3_12: the stored procedure master_dev._get_rule_dids_execute_cached() is used by the backend to evalute these rules, leaving the group-to-did
mappings in these two tables
MariaDB [dynamic_rules]> select * from dynamic_rules.rule_dids_38;
+-----+--------+
| did | roa_id |
+-----+--------+
|   2 |      0 |
|   4 |      0 |
+-----+--------+
2 rows in set (0.00 sec)

MariaDB [dynamic_rules]> select * from dynamic_rules.rule_dids_39;
+-----+--------+
| did | roa_id |
+-----+--------+
|   3 |      0 |
|   4 |      0 |
+-----+--------+
2 rows in set (0.00 sec)

# FIXME_INFO_ISSUE_3_13: we create a new virtual device named vdev_type_c_type_d, then update the cache by calling
MariaDB [dynamic_rules]> call master_dev._get_rule_dids_execute_cached(39);
Query OK, 0 rows affected (0.01 sec)

# FIXME_INFO_ISSUE_3_14: and we see that the new device has been added
MariaDB [dynamic_rules]> select * from dynamic_rules.rule_dids_39;
+-----+--------+
| did | roa_id |
+-----+--------+
```

```
|   3 |      0 |
|   4 |      0 |
|   5 |      0 |
+-----+--------+
3 rows in set (0.00 sec)
```

Tables

master_dev.device_groups

```
CREATE TABLE `device_groups` (
  `dgid` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `dg_guid` char(32) DEFAULT NULL,
  `ppguid` char(32) DEFAULT NULL,
  `shared` tinyint(1) NOT NULL,
  `name` varchar(64) NOT NULL,
  `icon` varchar(128) DEFAULT NULL,
  `force_inheritance` tinyint(1) NOT NULL,
  `user_owner` int(10) unsigned NOT NULL,
  `user_edit` int(10) unsigned NOT NULL,
  `edit_date` datetime NOT NULL,
  PRIMARY KEY (`dgid`),
  UNIQUE KEY `dg_guid` (`dg_guid`),
  KEY `did` (`shared`),
  KEY `user_owner` (`user_owner`),
  KEY `ppguid` (`ppguid`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

```

This is where you would find device group names and ids.

master_dev.device_groups_to_devices

```
CREATE TABLE `device_groups_to_devices` (
  `dgid` int(10) unsigned NOT NULL,
  `did` int(10) unsigned NOT NULL,
  UNIQUE KEY `dgid_did` (`dgid`,`did`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

```

This is where you would find static members of device groups enumerated.

master_dev.device_groups_to_groups

```
CREATE TABLE `device_groups_to_groups` (
  `dgid` int(10) unsigned NOT NULL,
  `sub_dgid` int(10) unsigned NOT NULL,
  UNIQUE KEY `dgid_2` (`dgid`,`sub_dgid`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

I believe this is used to define a device group as a set of other device groups. If we had rows (1,2), (1,3), (1,4), it would mean the dg 1 contained the devices in groups 2, 3 and 4.

master_dev.device_groups_rules

```
CREATE TABLE `device_groups_rules` (
```

```
  `rule_id` int(11) NOT NULL AUTO_INCREMENT,
  `owner_type` tinyint(4) unsigned NOT NULL DEFAULT '0' COMMENT '1: device group (dgid), 2: service set (service_set_id)',
  `owner_id` int(11) NOT NULL,
  `TTL` int(11) NOT NULL DEFAULT 5 COMMENT 'In minutes',
  `last_calculated` datetime NOT NULL DEFAULT '0000-00-00 00:00:00',
  PRIMARY KEY (`rule_id`),
  KEY `owner_type` (`owner_type`,`owner_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

```

The TTL value is the time-to-live. It is the number of minutes between subsequent executions of the rule by the dynamic device group updater process.

master_dev.device_groups_rules_selectors

```
CREATE TABLE `device_groups_rules_selectors` (
  `rule_id` int(11) NOT NULL,
  `selector_domain` tinyint(4) NOT NULL DEFAULT '0' COMMENT '0: selector_type is from definitions_rule_selector_types, 1: selector_type is from master_custom.custom_rule_selector_types',
  `selector_type` int(11) NOT NULL COMMENT 'What data field this selector is searching on, listed in master.definitions_rule_selector_types',
  `selector_instance` int(11) NOT NULL DEFAULT '0' COMMENT 'A single selector is an implicit OR of all its targets; you can use separate selector instance IDs to make multiple instances of the same selector, ANDed together.',
  `selector_target` int(11) NOT NULL COMMENT 'integer selector target, for logic types 0, 1, and 3',
  `selector_pattern` varchar(255) NOT NULL COMMENT 'string selector pattern, for logic types 2 and 4',
  `inverted` tinyint(4) NOT NULL COMMENT '0: positive match, normal sense; 1: positive match, inverted sense; 2: negative match, normal sense; 3: negative match, inverted sense',
  KEY `rule_selector` (`rule_id`,`selector_domain`, `selector_type`,`selector_instance`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Selectors do the work of rules, specifying acceptance criteria for devices to be considered part of the group. For example, here is where you would find a pattern that matches on the device name.

I believe when selector_domain is 1, the table master_custom.custom_rule_selector_types is not used. I can't find it anywhere in the em7 core code repo. In procedures.sql, however, we see this:

```
chris@cmarshall_ubuntu1:~/home/projects/em7_git_repos/em7_v8/G3code/sql$ grep "custom" procedures.sql
      SET join_clause = CONCAT(join_clause,' LEFT JOIN master_custom.ent_device_custom_extended_string ',c_selector_table,' ON (ld.id = ',c_selector_table,'.ent_id AND ',c_selector_table,'.field_id = \'',c_selector_type,'\' AND ',c_selector_expr,')');
      SET join_clause = CONCAT(join_clause,' LEFT JOIN master_custom.ent_device_custom_extended_integer ',c_selector_table,' ON (ld.id = ',c_selector_table,'.ent_id AND ',c_selector_table,'.field_id = ',c_selector_type,' AND ',c_selector_expr,')');
```

```
    SET join_clause = CONCAT(join_clause,' LEFT JOIN master_custom.ent_device_custom edc ON (ld.id = edc.ent_id)');
```

I suspect that when a rule has selector_domain==1, the ent_device_custom_extended_{string,integer} tables are used.

master_custom.ent_device_custom_extended_string

```
CREATE TABLE `ent_device_custom_extended_string` (
  `ent_id` bigint(20) NOT NULL,
  `field_id` int(11) NOT NULL,
  `value` varchar(511) CHARACTER SET utf8 NOT NULL,
  PRIMARY KEY (`ent_id`,`field_id`),
  INDEX field (field_id)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

master_custom.ent_device_custom_extended_int

```
CREATE TABLE `ent_device_custom_extended_integer` (
  `ent_id` bigint(20) NOT NULL,
  `field_id` int(11) NOT NULL,
  `value` bigint(20) NOT NULL,
  PRIMARY KEY (`ent_id`,`field_id`),
  INDEX field (field_id)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

master_dev.device_groups_rules_depth_expansion

```
CREATE TABLE `device_groups_rules_depth_expansion` (
  `rule_id` int(11) NOT NULL,
  `select_descendants` tinyint(4) NOT NULL COMMENT '0: select ancestors (or self); 1: select descendants (or self)',
  `min_depth` int(11) NOT NULL DEFAULT '0',
  `max_depth` int(11) DEFAULT NULL,
  `max_xlinks` int(11) DEFAULT NULL,
  `xlink_pattern` varchar(255) DEFAULT NULL,
  KEY `rule_id` (`rule_id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

dynamic_rules.rule_dids_X

```
CREATE TABLE `rule_dids_0` (
    `did` INT UNSIGNED NOT NULL,
    `roa_id` INT UNSIGNED,
    PRIMARY KEY (`did`)
  ) ENGINE=InnoDB DEFAULT CHARSET=latin1;

```

These tables are the cache of dynamic device rule evaluations.

For each rule ID X, the table dynamic_rules.rule_dids_X has one row for the did of every device belonging to the rule.

master.definitions_rule_selector_types

Table definition:

```
CREATE TABLE `definitions_rule_selector_types` (
  `selector_type` int(11) NOT NULL,
  `descr` varchar(32) NOT NULL COMMENT 'Human-readable name for type',
  `schema` varchar(64) NOT NULL COMMENT 'db.table.field, for reference only',
  `logic_type` tinyint(4) NOT NULL COMMENT 'The type of SQL logic used for this selector type.  0: 1-to-1 integer match; 1: 1-to-1 bit math; 2: 1-to-1 string match; 3: many-to-1 integer match; 4: many-to-1 string match',
  `query_field` varchar(32) NOT NULL COMMENT 'Table alias and field used in the actual query',
  `php_filter` varchar(32) NOT NULL COMMENT 'DEPRECATED: The canonical PHP filter name associated with this selector type',
  PRIMARY KEY (`selector_type`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

```

and standard populated values:

```
ariaDB [(none)]> select * from master.definitions_rule_selector_types;
+---------------+---------------------+---------------------------------------+------------+---------------------+----------------+
| selector_type | descr               | schema                                | logic_type | query_field         | php_filter     |
+---------------+---------------------+---------------------------------------+------------+---------------------+----------------+
|             0 | Organization        | master_dev.legend_device.roa_id       |          0 | ld.roa_id           | roa_id         |
|             1 | Device Class        | master_dev.legend_device.class_type   |          0 | ld.class_type       | class_type     |
|             2 | Device Category     | master.definitions_dev_classes.family |          0 | dcl.family          | family         |
|             3 | Device Name         | master_dev.legend_device.device       |          2 | ld.device           | device         |
|             4 | Device IP           | master_dev.device_ip_addr.ip          |          4 | dia.ip              | multi_ip       |
|             5 | Device State        | master_dev.device_state.state         |          0 | IFNULL(dst.state,0) | state          |
|             6 | Collection State    | master_dev.legend_device.active       |          1 | ld.active           | active_bitwise |
|             7 | Collector Group     | master_dev.legend_device.cug_id       |          0 | ld.cug_id           |                |
|             8 | Open TCP Ports      | master_dev.device_ports.port_num      |          3 | dp.port_num         | port_num       |
|             9 | Running Process     | master_dev.device_processes.name      |          4 | dproc.name          | procname       |
|            10 | Windows Service     | master_dev.device_services.name       |          4 | dserv.name          | servname       |
|            11 | Subscribed Product  | master_biz.product_skus_aligned.sku_id |          3 | psa.sku_id          | sku_id         |
|            12 | Active Event        | master_events.events_active.etype     |          3 | ea.etype            | etype          |
|            13 | Aligned Dynamic App | master.dynamic_app_collection.app_id  |          3 | dac.app_id          |                |
|            20 | Asset Make          | master_biz.legend_asset.make          |          2 | la.make             | make           |
```

```
|            21 | Asset Model         | master_biz.legend_asset.model          |          2 | la.model            | model          |
|            22 | Asset Function      | master_biz.legend_asset.function       |          2 | la.function         | function       |
|            23 | Asset Owner         | master_biz.legend_asset.owner          |          2 | la.owner            | owner          |
|            24 | Asset Location      | master_biz.asset_location.location     |          2 | al.location         | location       |
|            25 | Asset Serial        | master_biz.legend_asset.serial         |          2 | la.serial           | serial         |
|            26 | Asset Tag           | master_biz.legend_asset.asset_tag      |          2 | la.asset_tag        | asset_tag      |
|            40 | Software Title      | master_biz.asset_software.s_product    |          4 | asoft.s_product     | s_product      |
|            41 | Software Version    | master_biz.asset_software.s_version    |          4 | asoft.s_version     | s_version      |
+---------------+---------------------+----------------------------------------+------------+---------------------+----------------+
23 rows in set (0.000 sec)

```

Note from the comment on logic_type that:

- 0: 1-to-1 integer match
- 1: 1-to-1 bit math
- 2: 1-to-1 string match
- 3: many-to-1 integer match
- 4: many-to-1 string match

master.V_rule_selector_types.sql

I believe the idea behind this table is to extend master.definitions_rule_selector_types with custom types derived from customer specific data like custom attributes.

On my test stack, I created a single custom attribute (some_attr) and assigned a value to this attribute on a virtual device. Selecting from the table showed the same contents as master_definitions.rule_selector_types except for the last row, which is new:

```
select * from master.V_rule_selector_types;
+-----------------+---------------+---------------------+-----------------------------------+------------+---------------------+----------------+
| selector_domain | selector_type | descr               | schema                            | logic_type | query_field         | php_filter     |
+-----------------+---------------+---------------------+-----------------------------------+------------+---------------------+----------------+
|               0 |             0 | Organization        | master_dev.legend_device.roa_id   |          0 | ld.roa_id           | roa_id         |
|               0 |             1 | Device Class        | master_dev.legend_device.class_type |          0 | ld.class_type       | class_type     |
|               0 |             2 | Device Category     | master.definitions_dev_classes.family |          0 | dcl.family          | family         |
```

```
|               0 |             3 | Device Name         | master_dev.legend_device.device
|          2 | ld.device           | device         |
|               0 |             4 | Device IP           | master_dev.device_ip_addr.ip
|          4 | dia.ip              | multi_ip       |
|               0 |             5 | Device State        | master_dev.device_state.state
|          0 | IFNULL(dst.state,0) | state          |
|               0 |             6 | Collection State    | master_dev.legend_device.active
|          1 | ld.active           | active_bitwise |
|               0 |             7 | Collector Group     | master_dev.legend_device.cug_id
|          0 | ld.cug_id           |                |
|               0 |             8 | Open TCP Ports      | master_dev.device_ports.port_num
|          3 | dp.port_num         | port_num       |
|               0 |             9 | Running Process     | master_dev.device_processes.name
|          4 | dproc.name          | procname       |
|               0 |            10 | Windows Service     | master_dev.device_services.name
|          4 | dserv.name          | servname       |
|               0 |            11 | Subscribed Product  |
master_biz.product_skus_aligned.sku_id                |          3 | psa.sku_id          |
sku_id         |
|               0 |            12 | Active Event        | master_events.events_active.etype
|          3 | ea.etype            | etype          |
|               0 |            13 | Aligned Dynamic App |
master.dynamic_app_collection.app_id                  |          3 | dac.app_id          |
|
|               0 |            20 | Asset Make          | master_biz.legend_asset.make
|          2 | la.make             | make           |
|               0 |            21 | Asset Model         | master_biz.legend_asset.model
|          2 | la.model            | model          |
|               0 |            22 | Asset Function      | master_biz.legend_asset.function
|          2 | la.function         | function       |
|               0 |            23 | Asset Owner         | master_biz.legend_asset.owner
|          2 | la.owner            | owner          |
|               0 |            24 | Asset Location      | master_biz.asset_location.location
|          2 | al.location         | location       |
|               0 |            25 | Asset Serial        | master_biz.legend_asset.serial
|          2 | la.serial           | serial         |
|               0 |            26 | Asset Tag           | master_biz.legend_asset.asset_tag
|          2 | la.asset_tag        | asset_tag      |
|               0 |            40 | Software Title      |
master_biz.asset_software.s_product                   |          4 | asoft.s_product     |
s_product      |
|               0 |            41 | Software Version    |
master_biz.asset_software.s_version                   |          4 | asoft.s_version     |
s_version      |
|               1 |             1 | Device: some_attr   |
master_custom.ent_device_custom_extended_string.value |          2 | edces_1.value       |
|
+-----------------+---------------+---------------------+-----------------------------------
--------------------+------------+---------------------+----------------+
24 rows in set (0.002 sec)
```

This view references the custom attribute tables and is one reason rules can run slowly. Its create statement is:

```
/*!50001 CREATE VIEW `V_rule_selector_types` AS SELECT
 1 AS `selector_domain`,
 1 AS `selector_type`,
 1 AS `descr`,
```

```
 1 AS `schema`,
 1 AS `logic_type`,
 1 AS `query_field`,
 1 AS `php_filter`*/;
SET character_set_client = @saved_cs_client;
/*!50001 DROP VIEW IF EXISTS `V_rule_selector_types`*/;
/*!50001 SET @saved_cs_client          = @@character_set_client */;
/*!50001 SET @saved_cs_results         = @@character_set_results */;
/*!50001 SET @saved_col_connection     = @@collation_connection */;
/*!50001 SET character_set_client      = latin1 */;
/*!50001 SET character_set_results     = latin1 */;
/*!50001 SET collation_connection      = latin1_swedish_ci */;
/*!50001 CREATE ALGORITHM=UNDEFINED */

/*!50001 VIEW `V_rule_selector_types` AS SELECT 0 AS `selector_domain`, `drst`.`selector_type` AS `selector_type`, `drst`.`descr` AS `descr`, `drst`.`schema` AS `schema`,
`drst`.`logic_type` AS `logic_type`, `drst`.`query_field` AS `query_field`, `drst`.`php_filter` AS `php_filter` FROM `master`.`definitions_rule_selector_types` `drst`
UNION ALL SELECT 1 AS `selector_domain`, `edc`.`field_id` AS `selector_type`, Concat(CONVERT(Concat(Ucase(Substring(`edc`.`ent_class`, 1, 1)), Lcase(Substring(`edc`.`ent_class`, 2)) )
USING utf8), ': ', `edc`.`label`) AS `descr`, Concat('master_custom.ent_', CONVERT(`edc`.`ent_class` USING utf8), '_custom.', `edc`.`name`) AS `schema`, 2 AS `logic_type`,
Concat('edc.', `edc`.`name`) AS `query_field`, `edc`.`name` AS `php_filter` FROM `master_custom`.`ent_custom_fields` `edc` WHERE ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 0 )
 UNION ALL SELECT 1 AS `selector_domain`, `edc`.`field_id` AS `selector_type`, Concat(CONVERT(Concat(Ucase(Substring(`edc`.`ent_class`, 1, 1)), Lcase(Substring(`edc`.`ent_class`, 2)) )
 USING utf8), ': ', `edc`.`label`) AS `descr`, 'master_custom.ent_device_custom_extended_integer.value' AS `schema`, 2 AS `logic_type`, CONCAT('edcei_', `edc`.`field_id`,'.value') AS `query_field`,
   '' AS `php_filter` FROM `master_custom`.`ent_custom_fields` `edc` JOIN `master_custom`.`ent_device_custom_extended_integer` `edci` USING ( `field_id`)
   WHERE ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 1 AND `edc`.`type` = 1 ) GROUP BY `edci`.`field_id` HAVING Count(`edci`.`field_id`) > 0
   UNION ALL SELECT 1 AS `selector_domain`, `edc`.`field_id` AS `selector_type`, Concat(CONVERT(Concat(Ucase(Substring(`edc`.`ent_class`, 1, 1)),
   Lcase(Substring(`edc`.`ent_class`, 2)) ) USING utf8), ': ', `edc`.`label`) AS `descr`, 'master_custom.ent_device_custom_extended_string.value' AS
   `schema`, 2 AS `logic_type`, CONCAT('edces_',field_id,'.value') AS `query_field`, '' AS `php_filter` FROM `master_custom`.`ent_custom_fields` `edc`
   JOIN `master_custom`.`ent_device_custom_extended_string` `edcs` USING ( `field_id`) WHERE ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 1 A
   ND `edc`.`type` = 2 ) GROUP BY `edcs`.`field_id` HAVING Count(`edcs`.`field_id`) > 0 */;
```

Formatting this so a person can read it:

```
/*!50001 CREATE VIEW `V_rule_selector_types` AS SELECT
 1 AS `selector_domain`,
 1 AS `selector_type`,
 1 AS `descr`,
 1 AS `schema`,
 1 AS `logic_type`,
 1 AS `query_field`,
 1 AS `php_filter`*/;
```

```
SET character_set_client = @saved_cs_client;
/*!50001 DROP VIEW IF EXISTS `V_rule_selector_types`*/;
/*!50001 SET @saved_cs_client          = @@character_set_client */;
/*!50001 SET @saved_cs_results         = @@character_set_results */;
/*!50001 SET @saved_col_connection     = @@collation_connection */;
/*!50001 SET character_set_client      = latin1 */;
/*!50001 SET character_set_results     = latin1 */;
/*!50001 SET collation_connection      = latin1_swedish_ci */;
/*!50001 CREATE ALGORITHM=UNDEFINED */

/*!50001 VIEW
`V_rule_selector_types` AS SELECT
        0 AS `selector_domain`,
        `drst`.`selector_type` AS `selector_type`,
        `drst`.`descr` AS `descr`,
        `drst`.`schema` AS `schema`,
        `drst`.`logic_type` AS `logic_type`,
        `drst`.`query_field` AS `query_field`,
        `drst`.`php_filter` AS `php_filter`
FROM
        `master`.`definitions_rule_selector_types` `drst`

UNION ALL SELECT
        1 AS `selector_domain`,
        `edc`.`field_id` AS `selector_type`,
        Concat(
                CONVERT(
                        Concat(
                                Ucase(Substring(`edc`.`ent_class`, 1, 1)),
                                Lcase(Substring(`edc`.`ent_class`, 2))
                        )
                        USING utf8
                ), ': ', `edc`.`label`
        ) AS `descr`,
        Concat(
                'master_custom.ent_',
                CONVERT(`edc`.`ent_class` USING utf8),
                '_custom.',
                `edc`.`name`
        ) AS `schema`,
        1 AS `logic_type`,
        Concat('edc.', `edc`.`name`) AS `query_field`,
        `edc`.`name` AS `php_filter`
FROM
        `master_custom`.`ent_custom_fields` `edc`
WHERE
        ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 0 )
UNION ALL SELECT
        1 AS `selector_domain`,
        `edc`.`field_id` AS `selector_type`,
        Concat(
                CONVERT(
                        Concat(
                                Ucase(Substring(`edc`.`ent_class`, 1, 1)),
                                Lcase(Substring(`edc`.`ent_class`, 2))
                        ) USING utf8
                ), ': ', `edc`.`label`
        ) AS `descr`,
```

```
        'master_custom.ent_device_custom_extended_integer.value' AS `schema`,
        2 AS `logic_type`,
        CONCAT('edcei_', `edc`.`field_id`,'.value') AS `query_field`,
        '' AS `php_filter`
FROM
        `master_custom`.`ent_custom_fields` `edc`
JOIN
        `master_custom`.`ent_device_custom_extended_integer` `edci` USING ( `field_id`)
WHERE
        ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 1 AND `edc`.`type` = 1 )
GROUP BY
        `edci`.`field_id`
HAVING
        Count(`edci`.`field_id`) > 0
UNION ALL SELECT
        1 AS `selector_domain`,
        `edc`.`field_id` AS `selector_type`,
        Concat(CONVERT(Concat(Ucase(Substring(`edc`.`ent_class`, 1, 1)),
Lcase(Substring(`edc`.`ent_class`, 2)) ) USING utf8), ': ', `edc`.`label`) AS `descr`,
        'master_custom.ent_device_custom_extended_string.value' AS `schema`,
        2 AS `logic_type`,
        CONCAT('edces_',field_id,'.value') AS `query_field`,
        '' AS `php_filter` FROM `master_custom`.`ent_custom_fields` `edc`
FROM
        `master_custom`.`ent_custom_fields` `edc`
JOIN
        `master_custom`.`ent_device_custom_extended_string` `edcs` USING ( `field_id`)
WHERE
        ( `edc`.`ent_class` = 'device' AND `edc`.`extended` = 1 AND `edc`.`type` = 2 )
GROUP BY
        `edcs`.`field_id`
HAVING
        Count(`edcs`.`field_id`) > 0 */;

```

stored procedures

You can find a list of stored procedures here: Device group procedures

master_dev._get_rule_dids_execute_cached()

This is called by the dynamic device updater process. It evaluates a single dynamic device group rule to determine which dids belong to that rule's group, storing the results for rule_id X in dynamic_rules.rule_dids_X.

As this procedure it running, in makes inserts that can vary widely with selector type. Here is an example of the insert query for an IP address selector:

```
INSERT IGNORE INTO
        dynamic_rules.rule_dids_2529
SELECT
        ld.id AS did, ld.roa_id as roa_id
FROM
        master_dev.legend_device ld
LEFT JOIN
```

```
        master_dev.device_ip_addr `r2529s0_dia` ON (ld.id = `r2529s0_dia`.did AND (`r2529s0_dia`.ip LIKE '172.16.%'))
WHERE (`r2529s0_dia`.ip IS NOT NULL) GROUP BY ld.id

```

dynamic device updater process

This process run this query to get a list of rule id's:

```
        sql = """SELECT rule_id FROM master_dev.device_groups_rules dgr
                 WHERE dgr.last_calculated < DATE_SUB(NOW(), INTERVAL dgr.TTL MINUTE)"""
```

and then calls the stored procedure master_dev._get_rule_dids_execute_cached() on each rule_id

Post Mortem Articles

2024-08-22 EM-66517: MF rows behind fluctuating between 500k and 600k, found 4 DROP TEMPORARY TABLE IF EXISTS queries stuck for a long time that could be the cause of rows behind

ts_guide_to_jira_link

Link: 💔EM-66517: MF rows behind fluctuating between 500k and 600k, found 4 DROP TEMPORARY TABLE IF EXISTS queries stuck for a long time that could be the cause of rows behind DONE

From the description:

```
Observed Behavior:
MF rows behind fluctuating between 500k and 600k, found 4 DROP TEMPORARY TABLE IF EXISTS queries stuck for a long time that could be the cause of rows behind

Expected Behavior:
DROP TEMPORARY TABLE IF EXISTS queries shouldn't be stuck

Summary of troubleshooting steps taken so far and the current status:

The below DROP TEMPORARY TABLE IF EXISTS queries are stuck for a long time:

58238083        clientdbuser    10.152.2.132:60744      master_dev      Query   501595  NULL DROP TEMPORARY TABLE IF EXISTS master_dev._temp_dgroup_map
57690801        clientdbuser    10.152.2.132:35320      master_dev      Query   501595  NULL DROP TEMPORARY TABLE IF EXISTS master_dev._get_group_dids_result
57632960        clientdbuser    10.152.2.132:55374      master_dev      Query   501595  NULL DROP TEMPORARY TABLE IF EXISTS master_dev._temp_dgroup_map
57362989        clientdbuser    10.152.2.132:36200      master_dev      Query   501595  NULL DROP TEMPORARY TABLE IF EXISTS master_dev._temp_dgroup_map

The customer experienced this issue before https://sciencelogic.slack.com/archives/C8XHFKRB7/p1721930211809919 and sre failed over the RDS which fixed the issue, reached out to sre again https://sciencelogic.slack.com/archives/C8B2JLWMR/p1724332264576419 and they've requested an escalation.
```

```

I've tried to analyze the queries tables but I get a table don't exist warning.
```

2024-08-22 EM-66519: Parent device groups made up of dynamic child groups only show the first page of group members

ts_guide_to_jira_link

Link: 💔 EM-66519: Parent device groups made up of dynamic child groups only show the first page of group members TO DO

From the description:

```
Observed Behavior:
Expanding Device Group membership on the Registry->Device Groups page only shows the first page of results when a device group is made up of child dynamic device groups.
As far as I can tell, this only occurs for this type of "Parent" device group.

Expected Behavior:
All group members should be loaded and viewable in the UI

Summary of troubleshooting steps taken so far and the current status:
I have replicated the issue in a 12.2.0 support lab, accessible at 10.64.70.164 with em7admin:em7admin creds.
I created one simple dynamic device group named 00453572 Group 1 (Ping) with rules checking if the device class is Ping | ICMP:
That group only has 11 members, so all members are shown on one screen:
Then I created another simple dynamic group named 00453572 Group 2 (Not Ping) checking if the device class is NOT Ping | ICMP:
That group has 751 members, so it's paginated and properly shows all members:
Finally, I created a static group named 00453572 Parent that is just a "parent" group containing those two child groups:
This group properly displays the full first page of members no matter what your Page Result Count is set to, 25 in this example:

API Response from DevTools:
However, attempting to load any other page results in a complete failure:
And it produces an empty response without even any lookups defined:
{
"lookups": [],
"rows": [],
"totalRows": 0,
"page": 2,
"sort": "device",
"sortorder": null,
"search_queries": null,
"advanced_options": null,
"advanced_values": null,
"use_severity_badges": false,
"rowids": [],
"info_msg": [],
"err_msg": [],
"exec_js": null
}
Captured DevLog:
FWIW, the behavior is the same whether you're in AP2 or Classic
```

2024-08-15 EM-66436: Device Group ID xxx does not have a populated rule query (rule ID xxx)

ts_guide_to_jira_link

Link: EM-66436: Device Group ID xxx does not have a populated rule query (rule ID xxx)

CLOSED

From the description:

```
Observed Behavior: Device Group ID xxx does not have a populated rule query (rule ID xxx)

Expected Behavior: Resolve the issue

Summary of troubleshooting steps taken so far and the current status:
Datacom reached out regarding below errors:

Line 11692: Device Group ID 463 does not have a populated rule query (rule ID 292), so it cannot match on
    Line 11698: Device Group ID 464 does not have a populated rule query (rule ID 310), so it cannot match on
    Line 11701: Device Group ID 465 does not have a populated rule query (rule ID 317), so it cannot match on
    Line 11707: Device Group ID 467 does not have a populated rule query (rule ID 324), so it cannot match on
    Line 11710: Device Group ID 468 does not have a populated rule query (rule ID 331), so it cannot match on
    Line 11713: Device Group ID 469 does not have a populated rule query (rule ID 338), so it cannot match on
    Line 11719: Device Group ID 470 does not have a populated rule query (rule ID 345), so it cannot match on
    Line 11722: Device Group ID 476 does not have a populated rule query (rule ID 364), so it cannot match on
    Line 11725: Device Group ID 477 does not have a populated rule query (rule ID 371), so it cannot match on
    Line 11728: Device Group ID 478 does not have a populated rule query (rule ID 378), so it cannot match on
    Line 11731: Device Group ID 479 does not have a populated rule query (rule ID 385), so it cannot match on
    Line 11737: Device Group ID 480 does not have a populated rule query (rule ID 392), so it cannot match on
    Line 11740: Device Group ID 481 does not have a populated rule query (rule ID 399), so it cannot match on
    Line 11758: Device Group ID 538 does not have a populated rule query (rule ID 428), so it cannot match on
    Line 11764: Device Group ID 551 does not have a populated rule query (rule ID 440), so it cannot match on
    Line 11914: Device Group ID 1588 does not have a populated rule query (rule ID 804), so it cannot match on
    Line 11959: Device Group ID 2311 does not have a populated rule query (rule ID 885), so it cannot match on
    Line 12007: Device Group ID 3448 does not have a populated rule query (rule ID 1167), so it cannot match on

After checking further this is associated with dg_analysis_datacom.log which was the output of the script device_group_analyzer.py. shared in below escalation:
https://sciencelogic.atlassian.net/browse/EM-65598
```

```

Datacom reviewed the device groups. however could not find the rule query screenshot attached:

Questions from Datacom:
Can this affect our platform performance and what can be done to clean-up these Device Groups?

Reason for escalation:

Suggested by Randy:
https://sciencelogic.slack.com/archives/C07ACTERK55/p1723616755919989
```

The slack link is here: https://sciencelogic.slack.com/archives/C07ACTERK55/p1723616755919989 Connect your Slack account

2024-07-24 EM-66104: "Dev Count" column on Device Groups page is not accurate

ts_guide_to_jira_link

Link: 💔EM-66104: "Dev Count" column on Device Groups page is not accurate TO DO

From the description:

```
Observed Behavior:
On the Device Groups page, the "Dev Count" column is supposed to show the count of devices that are statically aligned with the device group.
However, in some cases this number shows a device count on groups where there is nothing statically aligned to the group.
A good example of this is device group ID 115:
I collected some DevLogs from the device groups page and identified the query that's being used to populate that number:

MySQL [(none)]> SELECT dgid, COUNT(*) dcount FROM master_dev.device_groups_to_devices WHERE dgid = 115 GROUP BY dgid;
+------+--------+
| dgid | dcount |
+------+--------+
|  115 |     83 |
+------+--------+
1 row in set (0.001 sec)

Sure enough, there are 83 devices that are still identified as attached to this group statically.

An excerpt of those devices:

MySQL [(none)]> SELECT * FROM master_dev.device_groups_to_devices WHERE dgid = 115;
+------+-------+
| dgid | did   |
+------+-------+
|  115 | 21614 |
|  115 | 21753 |
|  115 | 21754 |
|  115 | 21767 |
|  115 | 21807 |
|  115 | 21926 |
```

```
|  115 | 21927 |
|  115 | 22193 |
|  115 | 22194 |
|  115 | 22195 |
|  115 | 22196 |
|  115 | 22197 |
|  115 | 22209 |
....

What's even more confusing is that none of those devices even exist:

MySQL [(none)]> select device from master_dev.legend_device where ID in (21614,21753,21754,21767,21807,21926,21927,22193,22194,22195,22196,22197,22209,22211,22212,22213,22244,22245,22246,22247,22248,22249,22250,22251,22265,22266,22267,22268,22281,22536,22539,22540,22541,22542,23685,23686,23689,23690,24574,24575,25180,25181,25182,25636,26182,26183,26184,26185,26186,26187,26188,26189,26190,26191,26258,26259,26260,26261,26262,26263,26264,26265,26270,26271,26272,26273,26275,26645,26651,26658,26861,27118,27359,27369,27370,27372,28095,28540,30034,30035,30074,30075,30080);
Empty set (0.002 sec)

Expected Behavior:
Device count column should accurately reflect static device membership counts

Summary of troubleshooting steps taken so far and the current status:
I don't think there's any functional impact here since the dynamic membership is calculated accurately, and that's what matters for event suppression on dynamic groups, which is what the customer ultimately cares about.
However, I can't say for certain if there isn't some kind of performance impact from trying to do operations on devices that no longer exist.
I did find KB 6361 that explains how we could correct this situation:
https://support.sciencelogic.com/s/article/6361
This SQL command would clear out the database entries that are causing the bad counts to show:
delete from master_dev.device_groups_to_devices where did NOT in (select ID from master_dev.legend_device);
Unfortunately, that would only correct the counts as they stand right now.
It seems like there's a backend process that's not working as expected to clear out the necessary database entries when a device is deleted.
As a result, the device remains in the count even though it's no longer in the registry.
```

From this comment 💔 EM-66104: "Dev Count" column on Device Groups page is not accurate

TO DO

this query was used to check for non-existant devices in the device group:

```
SELECT dgtd.did
FROM master_dev.device_groups_to_devices as dgtd
left join master_dev.legend_device as ld on (ld.id=dgtd.did)
left join master_dev.legend_device_vanished as ldv on (ldv.id=dgtd.did)
WHERE
        dgid = 115 and
        isnull(ld.id) and
        isnull(ldv.id);
```

2024-04-23 EM-64567: Incorrect devices listed for Group-UI view and API Get Output

ts_guide_to_jira_link

Link: EM-64567: Incorrect devices listed for Group-UI view and API Get Output CLOSED

From the description:

```
Observed Behavior:
The device groups rules in master_dev.device_groups_rules didn't match what was defined in the UI until I manually ran the create_or_update_rule_query procedure against the rule. This caused a template to be applied to 2067 devices rather than the expected 72 devices in an example problem group, ID 99:
...
Before running the procedure, we can see that only the first rule was saved, which matched the 2067 devices:
MySQL [(none)]> select * from master_dev.device_groups_rules where owner_id=99\G;
*************************** 1. row ***************************
rule_id: 469
owner_type: 1
owner_id: 99
TTL: 5
last_calculated: 2024-04-05 19:09:37
rule_query: SELECT ld.id AS did, ld.roa_id as roa_id FROM master_dev.legend_device ld WHERE ((ld.class_type IN (4359,6461,4344,4378,4365,6641,4350,6454,4337,4371,4358,6460,4343,4377,4364,6590,4349,4383,4370,4357,6459,4342,4376,4363,6465,4348,4382,4369,4356,6458,4341,4375,4362,6464,4347,4381,4368,4354,6457,4340,4374,4361,6463,4346,4380,4367,4353,6456,4339,4373,4360,6462,4345,4379,4366,4352,6455,4338,4372))) GROUP BY ld.id
in_progress: 0
1 row in set (0.022 sec)

After running the procedure, the rules were corrected so only the expected 72 devices would match:
MySQL [(none)]> CALL `master_dev`.`create_or_update_rule_query`(469);
Query OK, 1 row affected (0.014 sec)

MySQL [(none)]> select * from master_dev.device_groups_rules where owner_id=99\G;*************************** 1. row ***************************        rule_id: 469     owner_type: 1
owner_id: 99
TTL: 5
last_calculated: 2024-04-05 19:09:37
rule_query: SELECT ld.id AS did, ld.roa_id as roa_id FROM master_dev.legend_device ld LEFT JOIN master_custom.ent_device_custom_extended_string edces_19 ON (ld.id = edces_19.ent_id AND edces_19.field_id ='19' AND (edces_19.value LIKE '%Microsoft%')) WHERE ((ld.class_type IN (4359,6461,4344,4378,4365,6641,4350,6454,4337,4371,4358,6460,4343,4377,4364,6590,4349,4383,4370,4357,6459,4342,4376,4363,6465,4348,4382,4369,4356,6458,4341,4375,4362,6464,4347,4381,4368,4354,6457,4340,4374,4361,6463,4346,4380,4367,4353,6456,4339,4373,4360,6462,4345,4379,4366,4352,6455,4338,4372)) AND (ld.cug_id IN (5)) AND (edces_19.value LIKE '%Microsoft%')) GROUP BY ld.id
in_progress: 0
1 row in set (0.001 sec)

Expected Behavior:

Device group rules should match what is set in the UI
```

2024-03-27 EM-64038: Dynamic Device group contains incorrect devices

ts_guide_to_jira_link

Link: EM-64038: Dynamic Device group contains incorrect devices CLOSED

From the description:

```
Observed Behavior: Dynamic device group rules shows number of matching devices 1499, the device group in the UI shows 5007 devices as well as running the query for the dynamic rule manually shows 5007 devices. The device rule is set to apply to all of the "Network." device categories however the device group is showing Servers as well. The device in question that lead to noticing this issue is a Windows Server 2008 R2 device id 1424.
```

2024-03-27 EM-64031: Substantial amount of long running SQL Queries in SLEEP and Locked Tables

ts_guide_to_jira_link

Link: EM-64031: Substantial amount of long running SQL Queries in SLEEP and Locked Tables CLOSED

I suspect that the Dynamic Device Group Updater process (DDGU) is causing widespread DB contention in this case.

From the description:

```
...
Observed Behavior: Long running sleep queries and locking observed occasionally resulting in sigterms
Expected Behavior: No locking should be present.
Summary of troubleshooting steps taken so far and the current status:
Customer has requested a RCA on why certain sleep queries are running for over 20000 seconds:
...
2024-03-24T23:31:00.914242+00:00 localhost silo.em7.1[7582]: ERROR proc_driver.subprocess_prune 888 Dynamic Device Groups Updater process did not complete in its allowed timeslot, SIGTERM'ing. Run time: 14m (max 13m)
2024-03-24T23:55:02.161791+00:00 localhost silo.em7.1[7582]: ERROR proc_driver.subprocess_prune 888 Enterprise Database: Asset Record Maintenance process did not complete in its allowed timeslot, SIGTERM'ing. Run time: 122m (max 120m)

dynamic_device_groups.log
...
```

2024-03-15 EM-63830 Query 9972328706 has been active for 290292 seconds (Threshold 900)

ts_guide_to_jira_link

Link: EM-63830: Query 9972328706 has been active for 290292 seconds (Threshold 900) CLOSED

From the description:

```
Observed Behavior:

The call (listed below) generates events in the events tab as it never seems to complete and it's process gets sigterm'ed.When manual execution of the call is attempted, the system responds that the call does not exist:

MariaDB [master_dev]> CALL master_dev.refresh_rule_selector_types();
ERROR 1305 (42000): PROCEDURE master_dev.refresh_rule_selector_types does not exist
This seems to potentially be related https://sciencelogic.atlassian.net/browse/SLUI-18634 opened by Robert Hoffman.

In addition, there are several Dyn Dev Grps which show 0 matched devices when Kent is certain that there are devices that should match and therefore be added to these groups. (See screenshots)
```

2024-02-22 EM-63460: Device Grouping and Event Suppression

ts_guide_to_jira_link

Link: EM-63460: Logical Disk Usage event not being created for Windows disk surpassing threshold CLOSED

From the description:

```
We are currently working on categorizing our servers into different groups within ScienceLogic, and we have observed some challenges with the functionality, particularly in relation to event suppression.

Could you please provide clarification on the following points:

Device Groups:
How does ScienceLogic define and manage device groups?
What is the intended functionality of adding servers to these groups?
Are there any specific considerations or best practices we should be aware of when utilizing device groups?

Event Suppression:
Can you explain the event suppression functionality within ScienceLogic?
Are there known issues or considerations when applying event suppression within the context of device groups?
What steps or best practices should we follow to ensure event suppression works seamlessly with our categorized servers?
We appreciate your assistance in helping us understand these functionalities better. Any additional documentation or resources you can share would also be highly beneficial.

Reason for Escalation: Engineering assistance needed to diagnose issue in order to find workaround
```

I suspect this is another case related to EM-60418.

2024-02-21 EM-63455: Event suppressions occactionally not working correctly

ts_guide_to_jira_link

Link: EM-63455: Event Suppressions occasionally not working correctly CLOSED

From the description:

```
Observed Behavior: The customer has defined a device group in order to suppress/mask the event ID 5198, but the events continue to trigger for devices in the device group.

Expected Behavior: The suppression should work and the event should not trigger.
```

@Randy Murdock thinks this is related to this fix: EM-60418: Events raised against devices in a device group are not suppressed as configured CLOSED

2024-02-21 EM-63432: events generating for devices in suppression (Dynamic device) group

ts_guide_to_jira_link

Link: EM-63432: Generating events for the devices are in suppression on Dynamic device group. CLOSED

From the description:

```
Observed Behavior: It seems that suppressed devices are not adhering to the suppression rules and are generating events randomly
Expected Behavior: Dynamic device group should suppress the all the alerts.
```

2024-01-25 EM-63055: Users losing access to device groups unexpectedly.

ts_guide_to_jira_link

Link: EM-63055: Users losing access to device groups unexpectedly. CLOSED

In this case, users with the correct permissions are randomly unable to view device groups for device groups with valid owners.

This known defect was thought to be related: EM-61101: Users unable to see shared device groups created by deleted users or suspended user accounts TO DO

2022-04-04 PQR BV Device Group Properties Page Timeout

Link: PQR BV Device Group Properties Page Timeout

This escalation demonstrates how to analyze the runtime of all rules executes in a run of DDGU.

2021-09-12 Chubb and Son: master_dev.get_assoc_group_dids_literal() called by UI with 100s of DGIDs to evaluate

Link: Chubb and Son: Dynamic Device Groups Query Performance | 2021 09 20 Analysis (Rob M) of Dynamic Device Group Rule Evaluation: why cant w...

In this escalation, the stored procedure master_dev.get_assoc_group_dids_literal() is being called by the UI with a list of hundreds of DGIDs to evaluate. Rob M wrote a great summary of

what the customer is trying to accomplish with their use of ddg's and how our implementation is performing poorly.

Known Issues

deadlocks with multi-row legend device updaters

See this post mortem section: [2020-04-23 Deadlocks: why these two queries can deadlock, strategies to avoid]

Silo Log Messages

master_dev.device_groups_to_devices

API doc

Device groups API documentation